



De La Salle University

College of Computer Studies

CSNETWK Laboratory #4: Web Server

Name (Last, First)	ID Number
Martinez, Gabrielle P.	12413267
Ortha, Gian Lorenzo C.	12414697
Tenorio, Jeroen Ralph I.	12415111
Xu, Kai Wen H.	12415375

1.0 Overview

This project demonstrates the fundamentals of socket programming and HTTP communication using Python. It focuses on building a simple web server capable of handling HTTP GET requests from a web browser. The server listens for incoming TCP connections, receives HTTP requests, parses the header, retrieves the requested HTML file from its local directory, and sends back an appropriate HTTP response message.

2.0 Design Approach

2.1 HTML (HelloWorld.html)

The HelloWorld.html file serves as the client-side resource requested from the web server. It is a simple HTML document placed in the same directory as the server scripts to test whether the server correctly retrieves and sends files in response to HTTP GET requests. The file contains basic HTML elements such as a <head> section for the page title and a <body> section displaying the text "Hello, World!" to confirm successful file transmission. When a web browser sends a request for HelloWorld.html to the server (for example, <http://127.0.0.1:6789/HelloWorld.html>), the server searches its local directory for the file, reads its contents, and sends it back to the client along with an HTTP response header. If the file is properly served, the browser displays its content, indicating that the server is functioning as expected. This HTML file plays a crucial role in validating the correct implementation of the server's request handling, file reading, and response construction mechanisms.

2.2 Server (webserver(single).py)

The webserver(single).py file implements the basic version of the web server that handles one client connection at a time. It demonstrates the foundational steps of socket programming, which include creating, binding, listening, and accepting connections. The server starts by creating a TCP socket using the socket module, binding it to a specific IP address and port number (e.g., port 6789), and listening for incoming connections. When a client sends an HTTP GET request, the server accepts the connection and receives the HTTP request message. It then parses the message to extract the requested filename, such as /HelloWorld.html. The server checks whether the file exists in its current directory. If it does, the server opens the file, reads its content, and constructs an HTTP response message containing a "200 OK" status line, appropriate headers, and the file's content. If the file cannot be found, the server instead sends a "404 Not Found" message with an HTML error

response. After sending the appropriate message, the server closes the client connection and waits for the next incoming request. This single-threaded implementation is straightforward and focuses on the sequential handling of client connections, providing a solid foundation for understanding how TCP sockets and HTTP protocols work together in web communication.

2.3 Concurrent Server (webserver(multi).py)

The `webserver(multi).py` file builds upon the functionality of the single-threaded version by introducing concurrency through multithreading. Instead of handling one client at a time, this version allows the server to process multiple client requests simultaneously, improving performance and responsiveness. The program uses Python's `threading` module to create a new thread each time a client establishes a connection. The main thread remains responsible for listening for incoming connections, while each spawned thread independently manages communication with a connected client—receiving the HTTP request, locating the requested file, sending the response, and closing the connection once completed. This design ensures that while one thread is serving a client, the main server can continue accepting new connections without blocking. As a result, multiple users can access different files at the same time, demonstrating how multithreading enhances scalability and efficiency in network-based applications. The multithreaded server not only highlights the importance of concurrency in modern web systems but also illustrates how Python can be used to design a responsive and efficient server capable of handling parallel client interactions.

3.0 Communication Flow

The communication process between the client and the server in this project follows the standard HTTP model. When a client, typically a web browser, initiates a connection to the server, it sends an HTTP GET request for a specific resource, such as `HelloWorld.html`, through a URL like `http://127.0.0.1:6789/HelloWorld.html`. This request includes the HTTP method, the requested file path, and additional header information identifying the host and other relevant details. Upon receiving this request, the server processes it by extracting the filename from the request line and searching for the corresponding file in its local directory. If the requested file is found, the server generates an HTTP response message containing a "200 OK" status line, followed by appropriate headers and the file's content as the message body. However, if the file is not present in the server's directory, it instead constructs an HTTP "404 Not Found" response along with an error message indicating that the requested resource could not be located.

4.0 Assumptions & Choices

The default port used is **6789**, but it can be changed in the server code as needed. The requested HTML file (e.g., `HelloWorld.html`) must be placed in the same directory as the server script for successful retrieval. The server includes a basic exception handler to manage missing files and unexpected client disconnections. The Python `threading` module is used for creating concurrent connections. The single-threaded version serves as the foundation for understanding basic socket operations before introducing concurrency. Only the HTTP GET method is supported, as this is sufficient for serving static web pages.

5.0 Execution Documentation

```
C:\Users\Kai Wen Xu\Downloads>python webserversingle.py
Ready to serve...
Connection established with ('192.168.31.248', 57489)
Ready to serve...
Connection established with ('192.168.31.248', 57481)
```

📶 Globe 📶

下午 5:49

🔋 75% 🔋

Hello, World!

This file is served by my Python web server!

大小

192.168.31.147

